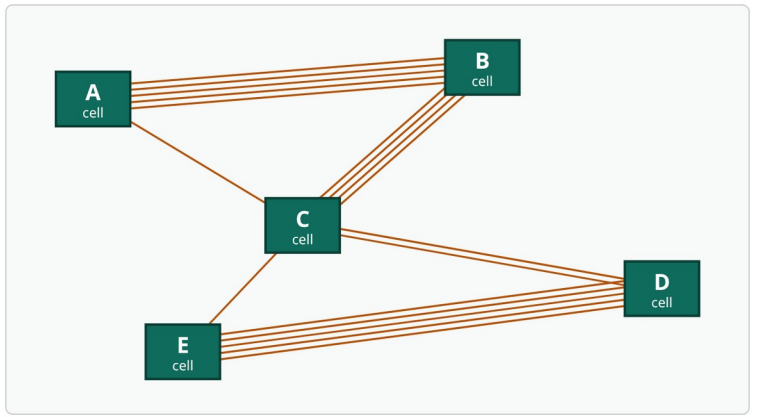
Initial bipartition

Before a computer can improve how a chip is split, it needs a first split, called the initial bipartition

From a chip to a graph

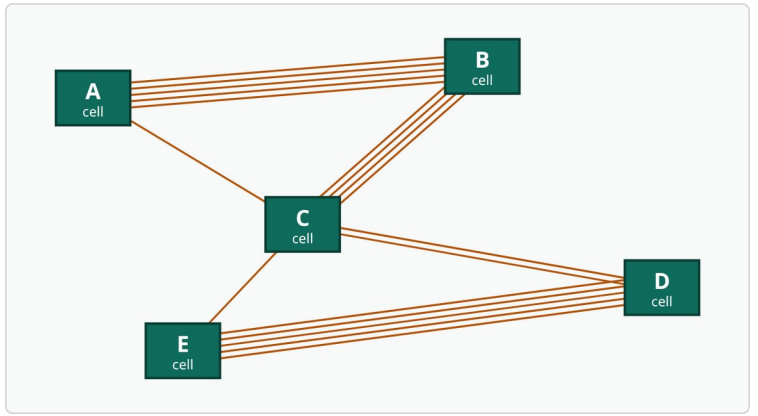
- A chip is made of small building blocks called cells, joined by wires
- Our example has five cells, named A to E
- Some pairs share many wires, others only one



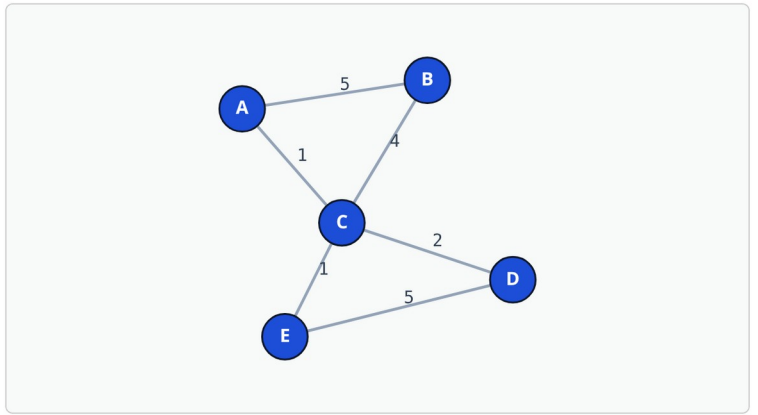
Cells become dots, wires become lines

- We draw each cell as a dot, called a node
- A line between two nodes is an edge
- Its number, the weight, counts the wires between the pair
- A–B and D–E are the heaviest, with five wires each

On the chip: cells joined by wires



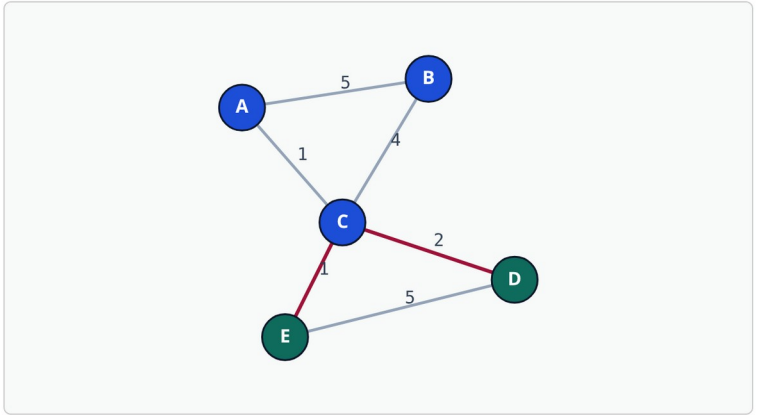
As a graph: weight = number of wires



Two numbers: cut and balance

- Splitting the cells into two sides is called a bipartition
- Edges that cross between the two sides are cut, and drawn in red
- The cutsize adds up their weights: here, two plus one makes three

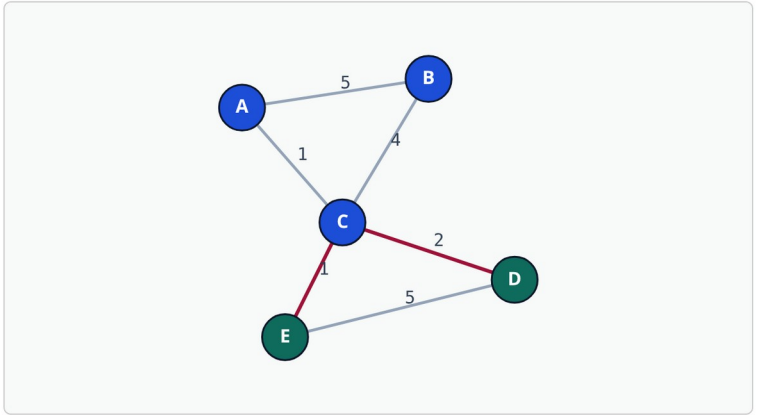
A B C | D E: cut C–D 2 + C–E 1 = 3. Sizes 3 and 2: legal.



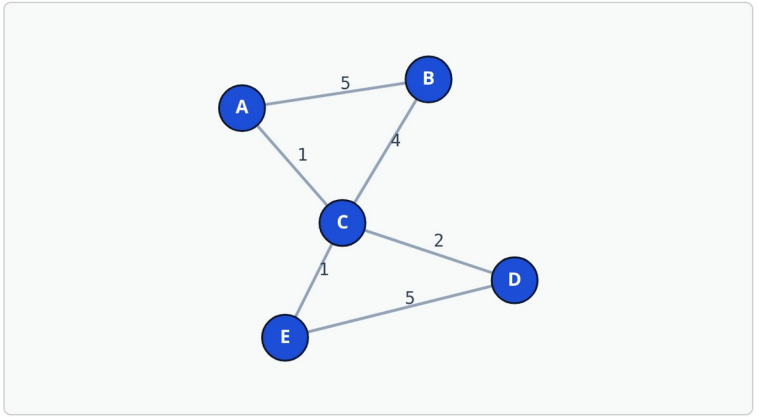
Balance: no side too big

- Putting every cell on one side gives a cut of zero, but that split is useless
- So we also check balance: how many cells each side holds
- A split is legal when no side holds more than three of the five

A B C | D E: cut C–D 2 + C–E 1 = 3. Sizes 3 and 2: legal.



All five on one side: cut 0. Sizes 5 and 0: not legal.



Why a starting split?

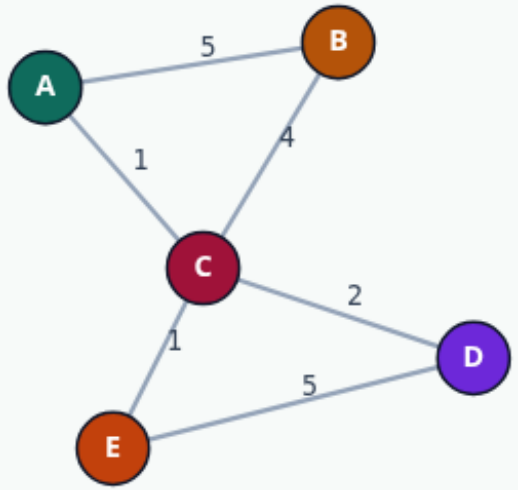
- Labs four and five teach two famous methods, called KL and FM
- They improve a split by moving cells from one side to the other
- But KL and FM need a legal split to start from
- Think of picking two teams: first any two teams, then swaps to make the game fair
- A good first split leaves less swapping to do
- Pause and guess: could a split made at random be as good as this one, with cut three?

Three ways to build a first split

INITIAL BIPARTITION

Three ways to build a first split

Step 1 / 5 · empty · module01-03-initial-bipartition



Explanation

Three short recipes build a first split: random, greedy and grow. Every node goes on one of two sides, and no side may hold more than 3 of the 5 nodes.

- Recipes: random, greedy, grow
- Every node on exactly one side
- Here: sides of 2 and 3 nodes

```
5 nodes, 6 edges, total weight 18
legal sizes: 2 | 3
```

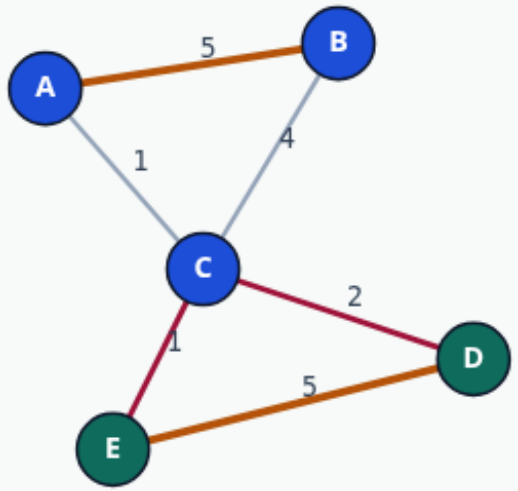
Three ways to build a first split

Random seed 5: lucky ABC|DE cut 3

INITIAL BIPARTITION

Random seed 5: lucky ABC|DE cut 3

Step 2 / 5 · random-lucky · module01-03-initial-bipartition



Explanation

Random shuffles the nodes like cards and deals two of them to one side. The seed fixes the shuffle. Seed 5 deals D and E: only C–D and C–E are cut, cutsize 3.

- Shuffle the names, deal 2 to one side
- Same seed, same split every time
- Seed 5 → ABC|DE, cut 3

```
method: random seed=5
parts: ABC|DE
cutsize: 3 = C–D 2 + C–E 1
```

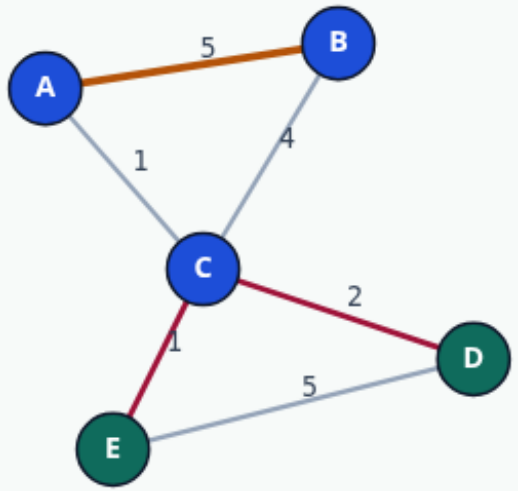
Random seed 5: lucky ABC|DE cut 3

Greedy: ABC|DE cut 3

INITIAL BIPARTITION

Greedy: ABC|DE cut 3

Step 3 / 5 · greedy · module01-03-initial-bipartition



Explanation

Greedy keeps the heaviest edge A–B inside one side (it ties with D–E; the smaller name wins). D, the node least tied to A and B, starts the other side. C joins A and B; E joins D, since that cuts 1 instead of 5.

- Anchor: A–B, weight 5
- Other side starts with D
- Each node goes where it adds less cut

```
method: greedy
parts: ABC|DE
cutsize: 3
```

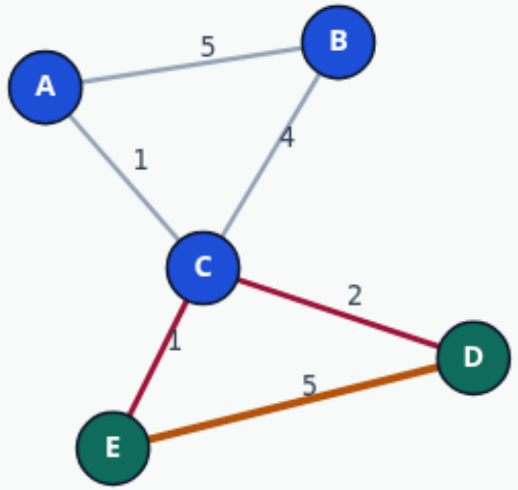
Greedy: ABC|DE cut 3

Grow from D: ABC|DE cut 3

INITIAL BIPARTITION

Grow from D: ABC|DE cut 3

Step 4 / 5 · grow-d · module01-03-initial-bipartition



Explanation

Grow starts a side with D and adds the node behind the heaviest edge leaving it: E, weight 5. The side now holds 2 of the 5 nodes, so it stops. A, B and C are the other side: cut 3.

- Start node: D
- Heaviest edge out: D–E (5)
- 2 of 5 nodes: stop

```
method: grow from D
parts: ABC|DE
cutsize: 3
```

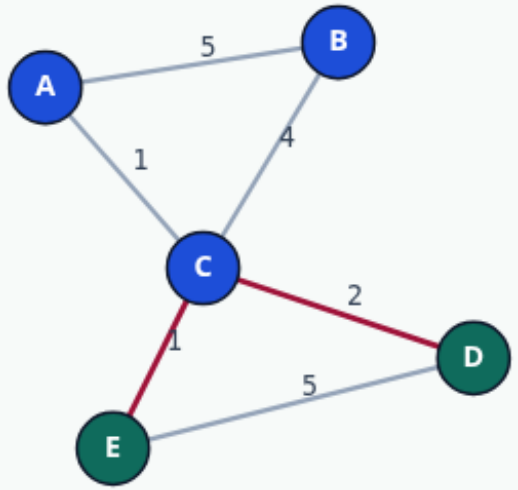
Grow from D: ABC|DE cut 3

All three found cut 3. Was that luck?

INITIAL BIPARTITION

All three found cut 3. Was that luck?

Step 5 / 5 · takeaway · module01-03-initial-bipartition



Explanation

Random seed 5, greedy and grow from D all give the same split, ABC|DE, with cut 3. Is that a good method, or luck?

- Random, seed 5 → cut 3
- Greedy → cut 3
- Grow from D → cut 3

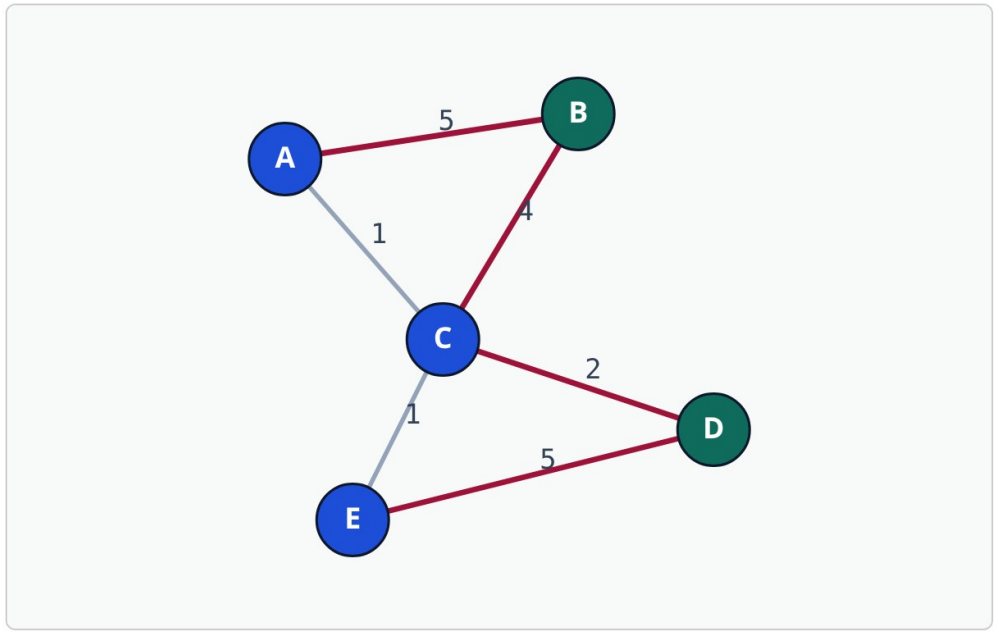
```
same split: ABC|DE, cut 3
luck, or a good method?
```

All three found cut 3. Was that luck?

Worked example: an unlucky seed

- A single unlucky shuffle can make the cut more than five times worse
- So yes, it was partly luck
- With seed three, B and D land on one side, and A, C and E on the other

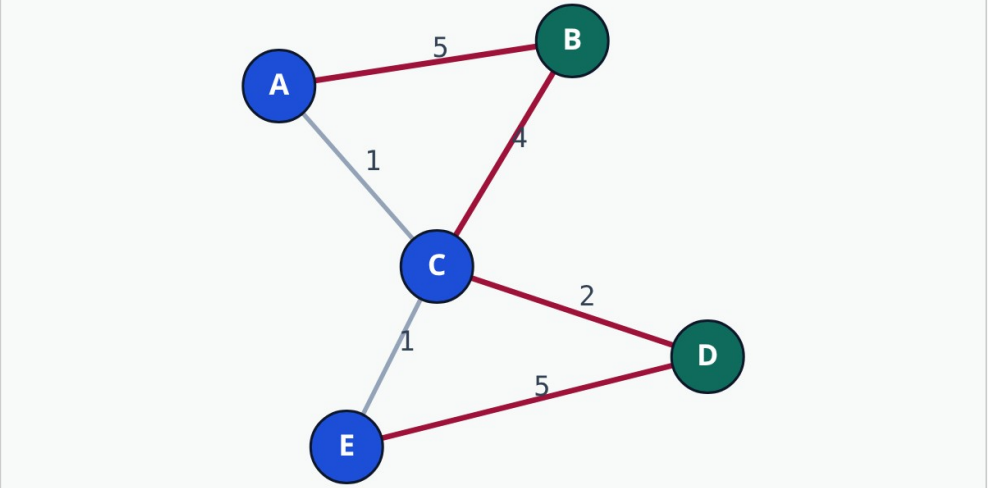
Random seed 3: A C E | B D



Adding up the cut

- Let's add up the cut, edge by edge
- A–B five, B–C four, C–D two, and D–E five
- That makes sixteen

Random seed 3: A C E | B D

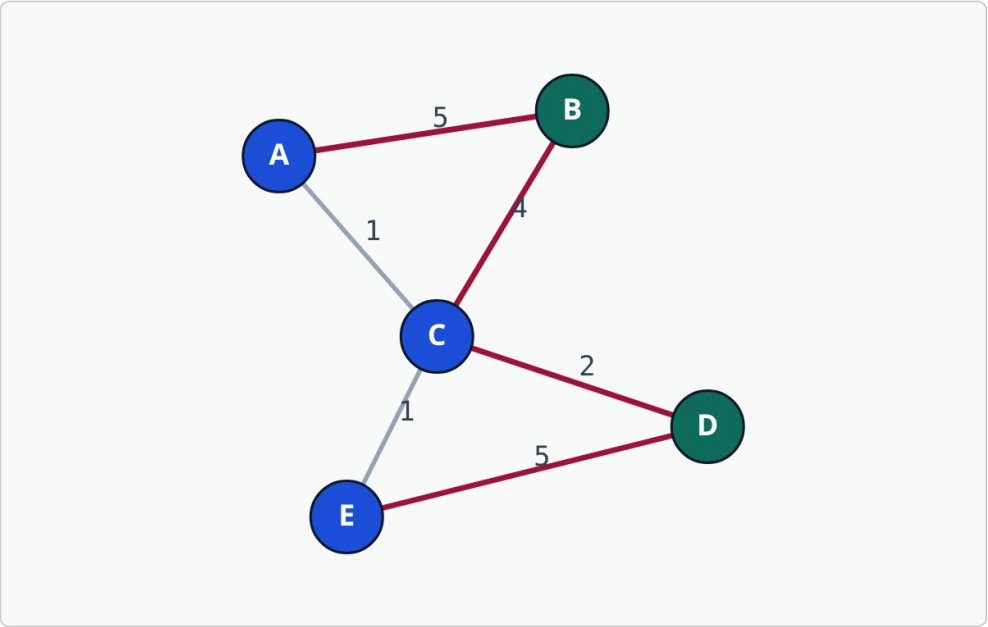



```
1. Cut (red): A–B, B–C, C–D, D–E
2. Cut: 5 + 4 + 2 + 5 = 16
```

The worst legal split

- Both heavy edges are cut, so this is the worst legal split of all
- Over sixteen seeds, the cut goes from three to sixteen
- Only three of those seeds hit the best cut, three

Random seed 3: A C E | B D

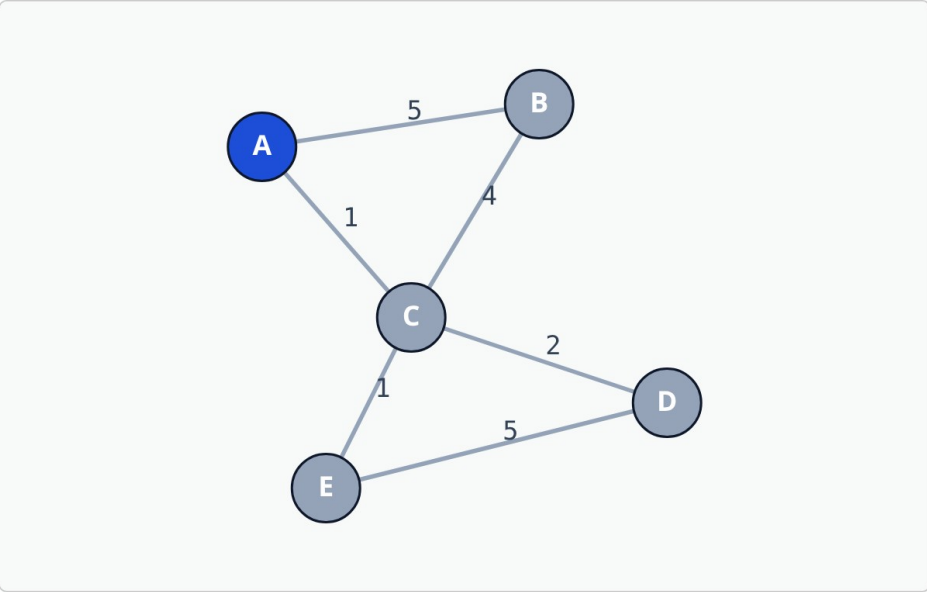



```
1. Cut (red): A–B, B–C, C–D, D–E
2. Cut: 5 + 4 + 2 + 5 = 16
3. Sizes 3 and 2: legal, but the worst start
```

Grow depends on the start node

- Grow has no luck in it, but it has a choice: the start node
- Let's start from A instead of D

Grow from A

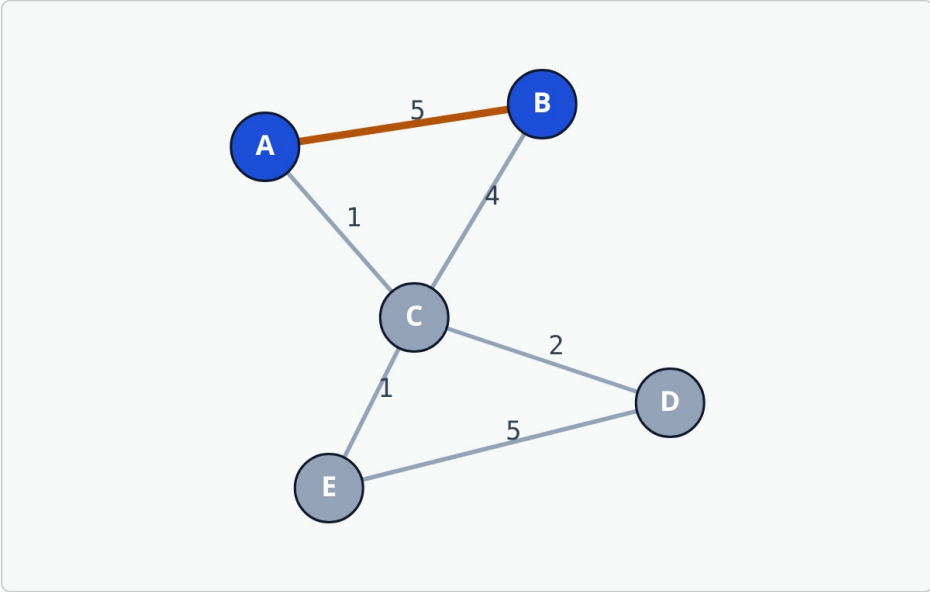



```
1. Start the side with A
```

Grow from A, step by step

- The heaviest edge leaving A goes to B, with weight five, so B joins A
- With two cells, the side is full, and it stops

Grow from A

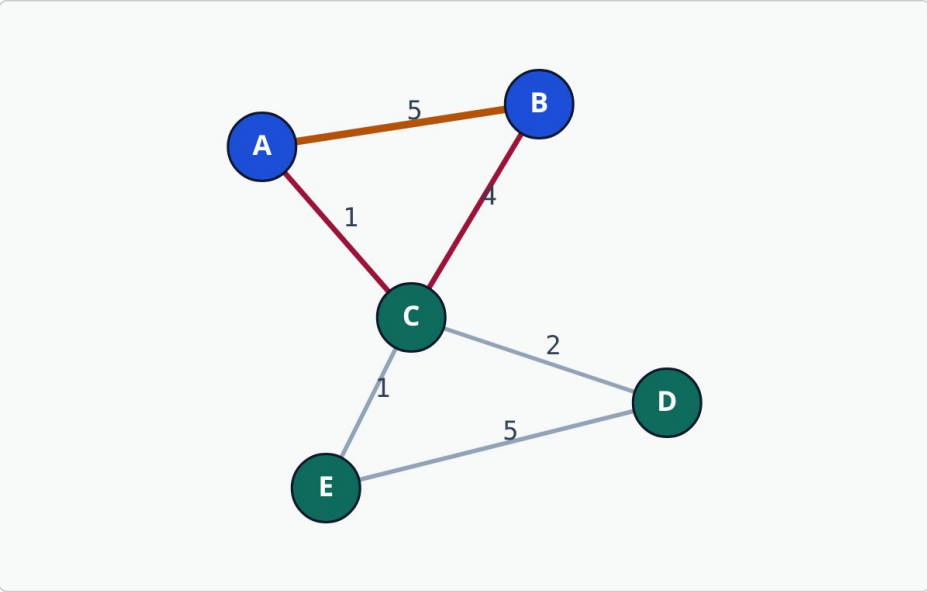



```
1. Start the side with A
2. Heaviest edge out of A: A–B 5, add B
3. 2 of 5 nodes: the side is full, stop
```

Grow from A: AB|CDE cut 5

- C, D and E form the other side, so A–C and B–C are cut
- One plus four makes five, which is worse than three
- Starting from C is even worse: a cut of nine

Grow from A




```
1. Start the side with A
2. Heaviest edge out of A: A–B 5, add B
3. 2 of 5 nodes: the side is full, stop
4. Cut: A–C 1 + B–C 4 = 5
```

Keep the best legal start

- The fix is simple: try several starts, keep the best
- Build a few random seeds, greedy, and grow from every node
- Drop any split that breaks the size rule, then keep the smallest cut
- Here, greedy and the five grows give three, five, five, nine, three and three
- So the best is three: A, B and C against D and E
- Pause and guess: can a smaller cut ever be the worse choice?

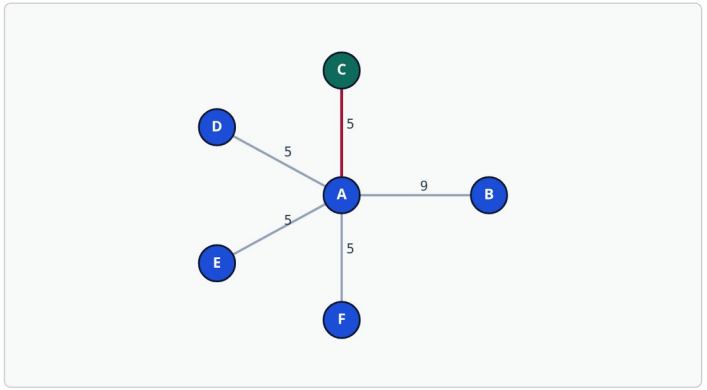
Keep the best legal start — try these

```
best_start(graph):
  starts = random(seed) for a few seeds
         + greedy
         + grow(x) for every node x
  drop every start that is not legal
  return the start with the smallest cutsize
```

Trap 1: no size limit

- Three traps can fool a method that looks sensible
- And yes, a smaller cut can be the worse choice
- Here is a star: A in the middle, wired to five other cells
- An older greedy had no size limit, so it kept adding cells next to A
- It ended with five cells against one, and a cut of five

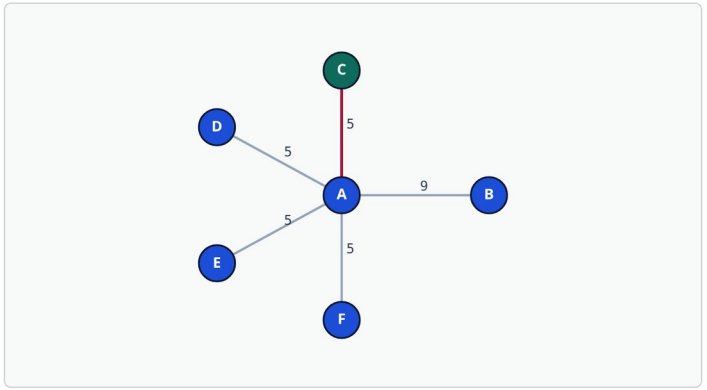
Old greedy, no size limit: A B D E F | C. Sizes 5 and 1, cut 5. Not legal.



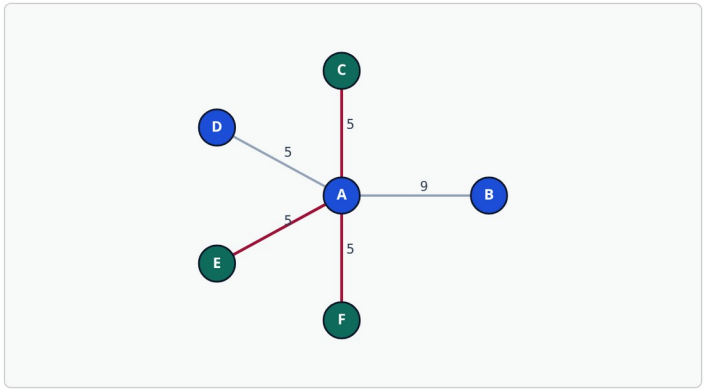
Trap 1: the size limit fixes it

- With six cells, each side must hold exactly three, so that split is not legal
- Our greedy stops each side at three cells, and gets fifteen
- No legal split of this star does better

Old greedy, no size limit: A B D E F | C. Sizes 5 and 1, cut 5. Not legal.



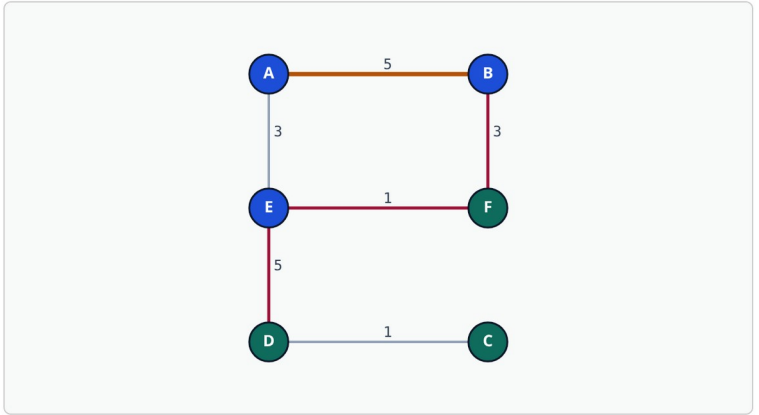
Greedy with the limit: A B D | C E F. Sizes 3 and 3, cut 15. Legal.



Trap 2: a tie can decide the answer

- In this graph, A–B and D–E both weigh five
- Greedy must pick one of them to start from
- The rule picks the smaller name, A–B, and that gives a cut of nine

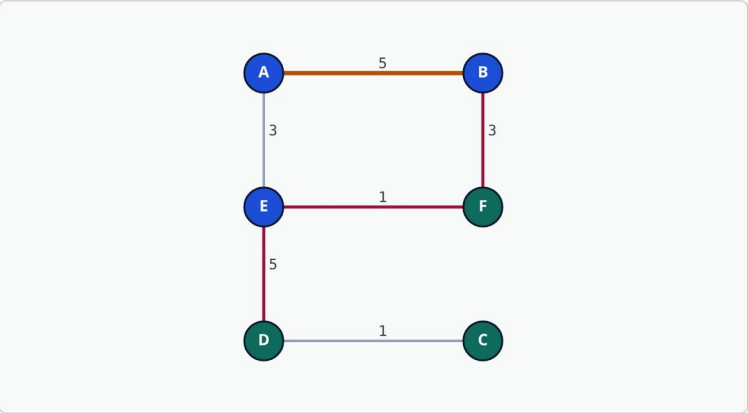
Anchor A–B (the rule): A B E | C D F, cut 9



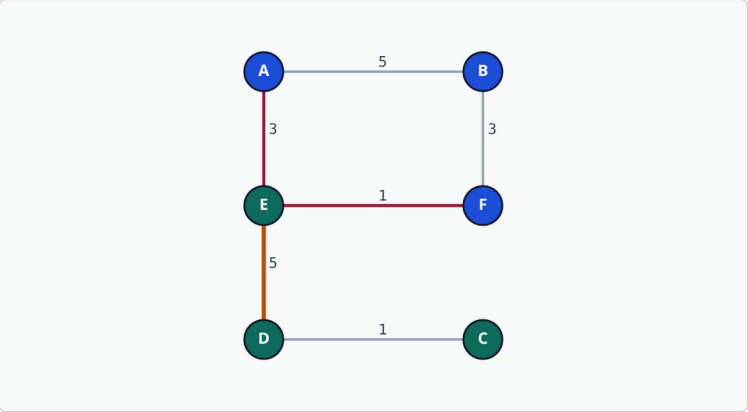
Trap 2: the other anchor

- Starting from D–E instead gives a cut of four
- So a tie rule more than doubled the cut
- Grow from C also finds four

Anchor A–B (the rule): A B E | C D F, cut 9

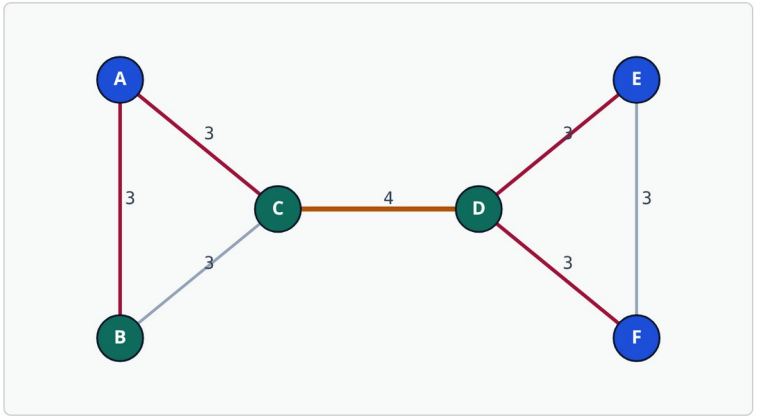


Anchor D–E: A B F | C D E, cut 4



Trap 3: the heaviest wire is not always inside

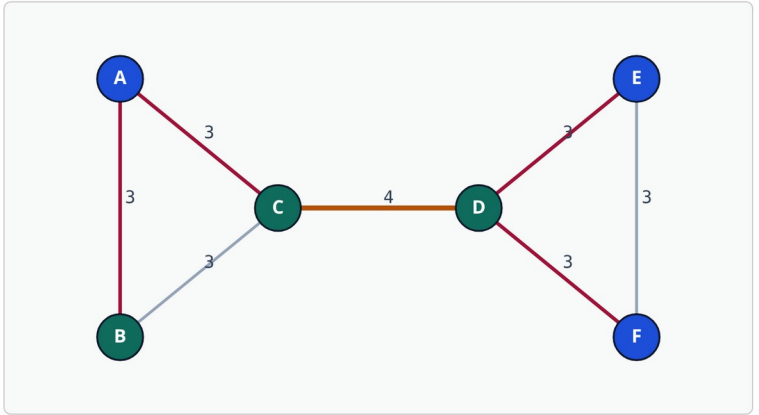
- Here are two triangles, joined by one bridge, C–D, with weight four
- Every triangle edge weighs three
- Greedy starts from the heaviest edge, which is the bridge itself
- So C and D begin on the same side, and the cut ends at twelve



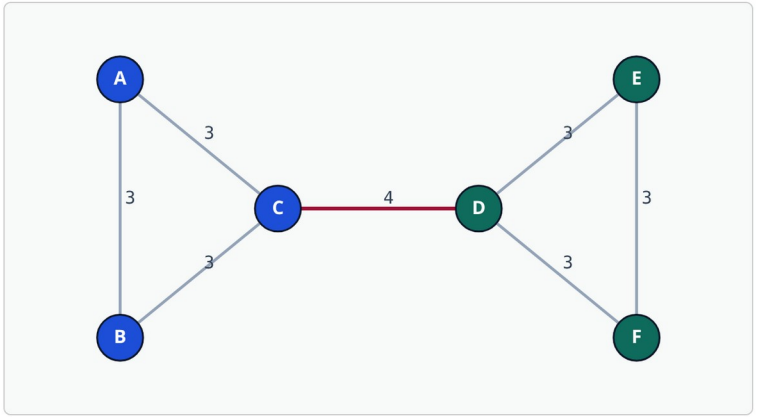
Trap 3: grow keeps the triangles

- Grow from A keeps its triangle together, and cuts only the bridge: four
- Pause and guess: is grow always better than greedy?

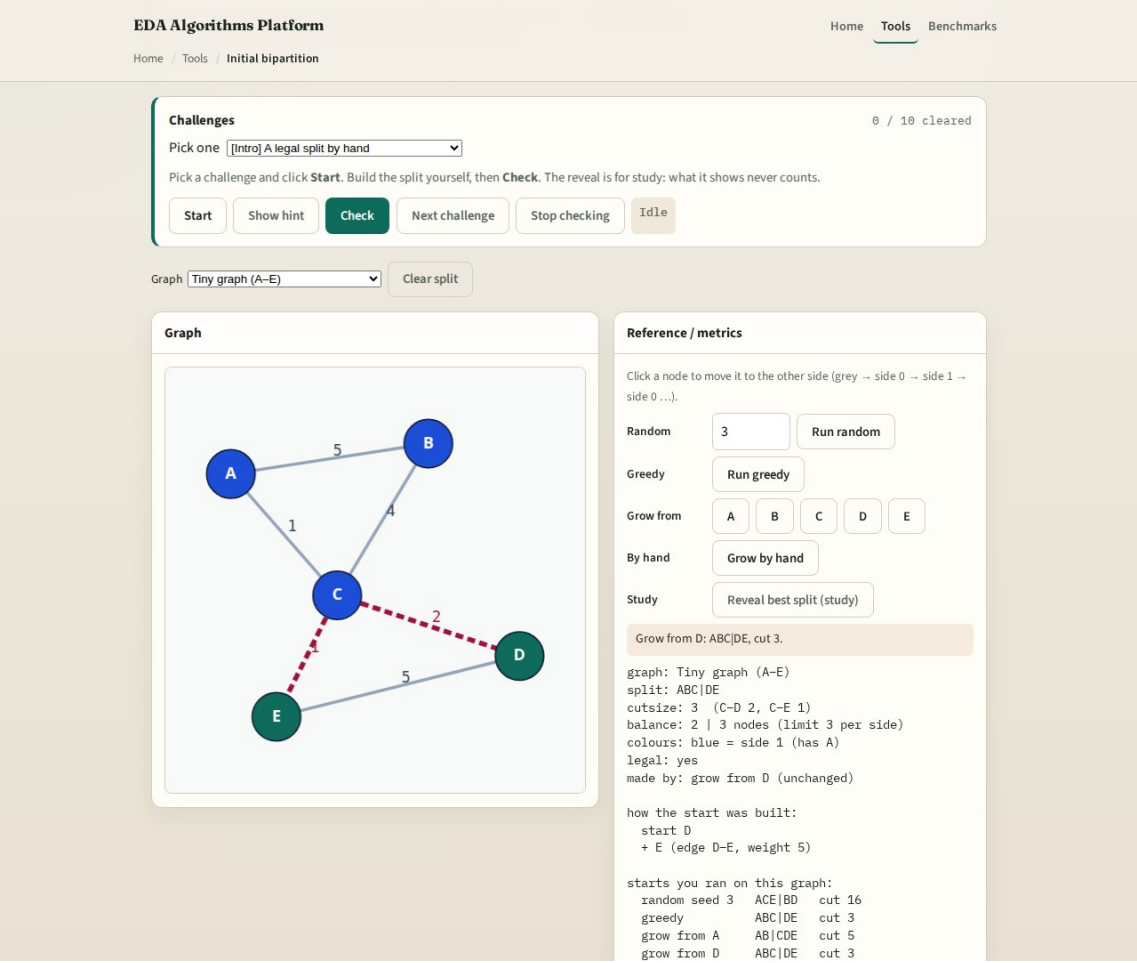
Greedy anchors the bridge C–D: A E F | B C D, cut 12



Grow from A: A B C | D E F, cut 4



Browser lab track



Browser lab starter

Implement track

wsl — module01-03-initial-bipartition/examples/.

```
# Track A — real Unix
# real shell session (Track A)

$ python3 initial_bipartition_check.py | grep PASS | head -14
PASS  tiny graph, random: seeds 0-15 give the golden splits and cuts
PASS  tiny graph, random: node order does not matter (input reversed)
PASS  tiny graph, random: every seed gives a legal 2|3 split
PASS  tiny graph, greedy: ABC|DE cut 3
PASS  star graph, greedy: ABD|CEF cut 15
PASS  tie graph, greedy: ABE|CDF cut 9
PASS  bridge graph, greedy: AEF|BCD cut 12
PASS  tiny graph, greedy: input reversed gives the same split
PASS  tie graph, greedy: input reversed gives the same split
PASS  star graph, greedy: both sides stay within ceil(n/2) nodes (3|3)
PASS  tiny graph, grow: from each node gives the golden split and cut
PASS  bridge graph, grow: from each node gives the golden split and cut
PASS  tiny graph, grow: input reversed gives the same splits
PASS  tiny graph, legality: everything on one side (cut 0) is NOT legal

$ python3 initial_bipartition_check.py | tail -1
all goldens match
```

Real shell session

Implement track — try these

```
# print every correct answer
python3 examples/initial_bipartition_check.py --show

# check your own functions
python3 examples/initial_bipartition_check.py --impl my_init.py
```

Remember

- Random is fast, but it depends on luck
- Greedy keeps heavy edges inside, but ties and bridges can fool it
- Grow depends on its start node
- Report the balance with every cut, and never let a side pass its limit
- Break ties by name, not by the order of the input file
- So try several starts, keep the best legal one

Your turn

- Pick a track: the browser lab, Python, or both
- Tick each box of the checklist, then take the short quiz
- Next comes Kernighan–Lin, which starts from your split and makes it better
- Pause and guess: how much can it shrink the unlucky cut of sixteen?
- Lab four will show you